CRITICAL OUTAGE // SEV-1

INCIDENT ID: NX-2026-9941 | **FACILITY:** Melbourne Hyperscale Core Exchange | **TARGET:** x86_64 Linux / C++20

PACKET RUSH: OPERATION DATA SHIELD

Part 1: The Infrastructure Crisis & Technical Challenge

1. Incident Briefing: The Melbourne Hyperscale Crisis

Following the commissioning of the high-density **Melbourne Hyperscale Core Exchange**, tier-1 carrier **NexusNet** activated its 400 Gbps multi-region backbone switching fabric. Connecting millions of simultaneous connections across international cloud transit paths, NexusNet's ingress edge switches are experiencing an unprecedented flood of traffic exceeding tens of millions of packets per second.

Security rule sets have ballooned to tens of thousands of active entries. Under current traffic surges and synchronized policy updates, the existing sequential rule-matching architecture has collapsed into **severe classification bottlenecks and ingress queue saturation**. Packets are queuing in NIC ring buffers, round-trip times are spiking, and downstream enterprise workloads are starving.

PRIMARY OBJECTIVE: Architect, implement, and benchmark a next-generation packet classification engine in C++20. Your engine must process high-velocity network traffic across a multi-dimensional **13-tuple header space** at line-rate speed (target: **25.0 Million packets/sec**), low batch latency (P99 batch latency ≤ 2.5 µs across 64-packet batches), within a strict **32.0 MiB** cache budget, supporting sub-5 microsecond dynamic updates.

2. The 13-Tuple Header Space & Predicates (Strict Conjunction)

NexusNet's data plane enforces policies across **13 fields** formatted as a 24-byte struct. A rule matches if and only if **all 13 predicates evaluate to true simultaneously (strict logical conjunction)**:

Field Name	Type & Width	Formal Matching Predicate	Wildcard Condition
`src_ip`	uint32_t (4B)	`(pkt.src_ip & mask) == (rule.src_ip.ip & mask)`	prefix_len == 0
`dst_ip`	uint32_t (4B)	`(pkt.dst_ip & mask) == (rule.dst_ip.ip & mask)`	prefix_len == 0
`src_port`	uint16_t (2B)	`pkt.src_port >= low && pkt.src_port <= high`	[0, 65535]
`dst_port`	uint16_t (2B)	`pkt.dst_port >= low && pkt.dst_port <= high`	[0, 65535]
`packet_len`	uint16_t (2B)	`pkt.packet_len >= low && pkt.packet_len <= high`	[20, 65535]
`region_id`	uint16_t (2B)	`rule.region_id.wildcard || pkt.region_id == rule.region_id.value`	wildcard == true
`vlan_id`	uint16_t (2B)	`rule.vlan_id.wildcard || pkt.vlan_id == rule.vlan_id.value`	wildcard == true
`protocol`	uint8_t (1B)	`rule.protocol.wildcard || pkt.protocol == rule.protocol.value`	wildcard == true
`tcp_flags`	uint8_t (1B)	`(pkt.tcp_flags & mask) == (rule.val & mask)`	mask == 0x00
`ingress_iface`	uint8_t (1B)	`rule.ingress_iface.wildcard || pkt.ingress_iface == rule.ingress_iface.value`	wildcard == true
`dscp`	uint8_t (1B)	`rule.dscp.wildcard || pkt.dscp == rule.dscp.value`	wildcard == true
`icmp_type`	uint8_t (1B)	`rule.icmp_type.wildcard || pkt.icmp_type == rule.icmp_type.value`	wildcard == true (Type 0 valid)
`ttl`	uint8_t (1B)	`pkt.ttl >= low && pkt.ttl <= high`	[0, 255]

• **Packet Field Validity Guarantee**: Harness guarantees all packets are structurally valid (packet_len ∈ [20, 65535], vlan_id ∈ [0, 4095], dscp ∈ [0, 63]). No ingress packet validation required.

• **TCP Flags & ICMP Independence**: All 8 bits supported (0x00–0xFF, including CWR 0x80 and ECE 0x40). icmp_type is evaluated strictly as an ordinary 8-bit field independently of protocol.

• **CIDR Mask Generation**: Input prefix signed int32_t (0 ≤ prefix_len ≤ 32). Bitmask generated via uint32_t unsigned arithmetic: static_cast<uint32_t>(~((uint32_t{1} << (32 - L)) - uint32_t{1})).

3. Priority Resolution & Global Ordering Invariant

• **Numerical Priority**: Lower numerical values indicate strictly higher priority (Priority 1 overrides Priority 10).

- **Deterministic Tie-Breaking & RuleId Reuse**: If two matching rules share numerical priority, the rule with the **lower rule_id** wins. A RuleId may be reused after its previous rule has been successfully deleted.
- **Global Ordering Invariant**: The winning rule must strictly minimize (priority, rule_id) globally across all matching candidates. Return Action::DROP if candidate set is empty; return winning rule action exclusively.
- **Default Action**: If no rule matches a packet, the classifier must return **Action::DROP**.
- **Actions**: ACCEPT (1), DROP (0), REJECT (2), REDIRECT (3).